

Учебный проект на Go

Маркетплейс: полное задание на реализацию

Цель	Реализуйте backend с пользователями, кошельками, каталогом, корзинами, заказами, промокодами, историей и фоновой обработкой. Сразу используйте структуры, PostgreSQL, HTTP API и тесты.
-------------	---

Это единое задание по программе V1-V17. Все возможности входят в итоговую реализацию, включая дополнительные задания из V2. Промежуточные версии на отдельных мапах и последовательные переносы хранилища делать не требуется.

Что должно получиться

- Один Go-сервис с HTTP API и PostgreSQL. Все пользовательские данные сохраняются после перезапуска.
- Покупка и отмена заказа выполняются целиком в транзакции. Параллельные запросы не приводят к повторным списаниям, возвратам и отрицательным остаткам.
- Фоновые уведомления обрабатываются через ограниченную очередь и worker pool. Есть корректное завершение работы и восстановление незавершенных задач.
- Проект можно поднять по README, выполнить демонстрационный сценарий и запустить автоматические тесты.

Границы учебной системы

Один общий каталог и один склад, одна валюта - рубли. Деньги в кошельке учебные; пополнение имитируется запросом. Платежный провайдер, доставка, продавцы, frontend и gRPC не входят в согласованный объем. Уведомления записываются в отдельный журнал доставки, внешний почтовый сервис не нужен.

Для проверки принадлежности данных используйте учебный заголовок X-User-ID. Это идентификатор действующего пользователя, а не полноценная аутентификация. Административные операции защищаются отдельным X-Admin-Token из конфигурации. Такой режим предназначен для локальной демонстрации.

Как работать	Разрабатывайте по функциональным блокам и сразу проверяйте их тестами. Один документ не означает, что весь проект нужно написать за один подход.
---------------------	--

1. Общие правила и модели

- ID сущностей - положительные int64, генерируются сервером. Повторяющиеся ID запрещены ограничениями БД. Клиент не назначает ID при создании.
- Деньги - int64 в копейках и BIGINT в БД. float для денег не используйте. Проверяйте переполнение при сложении, умножении и вычислении скидки.
- Имя пользователя и название товара обрезаются по краям; после этого длина от 1 до 200 символов Unicode. Поиск по подстроке без учета регистра. Пустой поисковый запрос возвращает весь список.
- Цена положительная; баланс и остаток неотрицательные; количество позиции положительное. Временные метки храните в UTC.
- Ошибочный запрос не меняет бизнес-данные. Все списки имеют стабильный порядок и пагинацию: limit по умолчанию 20, от 1 до 100; offset >= 0. По умолчанию сортировка по ID по возрастанию, история - по времени и ID по убыванию.

Сущности и поведение

Модель	Что отвечает за нее
User, Wallet	Профиль отдельно от кошелька. Wallet: HasEnoughMoney, TopUp, Withdraw. Суммы > 0, отказ без изменения баланса.
Product	Название, цена, остаток. IsAvailable, AddStock, Reserve; проверка количества и переполнения. Reserve уменьшает остаток при покупке.
Cart, CartItem	Add, SetQuantity, Remove, Clear, IsEmpty. Повторное добавление увеличивает количество. Clear оставляет непустую ссылку на пустую коллекцию.
Order, OrderItem	Состав со снимками названий и цен, сумма, скидка, статус. IsPaid, IsCancelled, MarkCancelled; переход paid -> cancelled только один раз.
Остальные модели	WalletTransaction, OrderStatusHistory, OperationLog, PromoCode, PromoRedemption, NotificationJob.

Структуры, методы и копии

Создавайте модели через функции с валидацией и error. Изменяющие методы работают через указатель; методы чтения не меняют данные. Коллекции заказа копируются при создании и возврате наружу. Изменение исходного слайса, корзины или результата чтения не должно менять сохраненный заказ.

Локальный метод MarkCancelled меняет только модель. Полный возврат денег и остатков выполняет сервис отмены в транзакции. Создание модели заказа само по себе не означает успешную оплату: paid сохраняется только вместе с успешным checkout.

2. Пользователи, кошельки и каталог

Пользователи

- Создать пользователя с именем; одновременно создать кошелек с нулевым балансом и пустую корзину.
- Получить профиль по ID, получить список, найти пользователей по части имени, переименовать существующего пользователя.
- Удалить пользователя. Чтобы сохранить заказы и финансовую историю, используйте мягкое удаление: `deleted_at`, очистка корзины, запрет новых операций и исключение из обычного поиска.
- Удаление разрешено только при нулевом балансе и отсутствии заказов `paid`. Иначе верните конфликт. Повторное удаление возвращает 404. Исторические записи остаются связанными с пользователем.

Кошелек

- Получить баланс. Пополнить существующему активному пользователю на положительную сумму.
- При каждом пополнении, оплате и возврате сохранить движение кошелька: тип, сумму со знаком, баланс после операции, ссылку на заказ при наличии, время.
- Баланс изменяется вместе с записью движения в одной транзакции. Параллельные пополнения не теряются.
- Произвольный HTTP-метод списания не нужен: списание выполняется при покупке. Локальный метод `Withdraw` остается частью модели и проверяется тестами.

Каталог

- Создать товар с названием, ценой и начальным остатком; получить товар и список товаров; искать по части названия.
- Изменить название и цену. Изменения влияют на будущие покупки и расчет корзины, но не на существующие заказы.
- Установить абсолютный остаток ≥ 0 и отдельно пополнить склад на положительное количество. Операции защищены от одновременного `checkout` транзакционной блокировкой.
- Товар с нулевым остатком остается в каталоге и может находиться в корзине. Наличие проверяется при оформлении.
- Управление каталогом доступно с административным токеном. Пользовательские чтения каталога доступны без него.

Результат

Через API можно создать пользователя, пополнить кошелек, добавить и изменить товары. Все изменения видны после перезапуска сервиса.

3. Корзина и оформление заказа

Корзина пользователя

- Получить корзину с позициями, текущими названиями и ценами, количеством, суммой каждой позиции, subtotal, discount и total.
- Добавить существующий товар в положительном количестве. Если позиция уже есть, увеличить количество. Добавление не резервирует склад и не списывает деньги.
- Установить количество существующей позиции: положительное значение заменяет количество, 0 удаляет позицию, отрицательное значение - ошибка. Для отсутствующей позиции верните 404.
- Удалить позицию; при ее отсутствии вернуть 404. Очистить корзину вместе с выбранным промокодом. Повторная очистка пустой корзины успешна.
- При примененном промокоде пересчитывать скидку на чтении и checkout. Если промокод стал недоступен, вернуть конфликт с понятной причиной; пользователь может удалить код.

Checkout: единая транзакция

Действующего пользователя определяйте по X-User-ID. Суммы, цены, владельца и статус заказа рассчитывает сервер; клиент передает только ключ идемпотентности в заголовке.

- Заблокировать необходимые записи, прочитать корзину, кошелек, товары и промокод. Проверить активность пользователя, непустую корзину, существование товаров, остатки и доступность промокода.
- Рассчитать subtotal по текущим ценам, discount и total. Проверить баланс. Сохранить новый заказ и позиции со снимками названий, цен и количества.
- Списать total с кошелька, уменьшить остатки, записать движение денег, использование промокода, статус paid и его историю.
- Очистить корзину и ее промокод. Добавить операцию checkout в историю пользователя и задачу уведомления. Зафиксировать все изменения одновременно.

При любой ошибке

Откатите всю транзакцию: баланс, остатки, корзина, заказы, движение денег, промокод, история и задача уведомления остаются как до запроса. Не отправляйте уведомление до commit.

Снимок заказа

После покупки изменение цены, названия товара или корзины не меняет состав, стоимость и скидку заказа. Просмотр заказа использует его сохраненные позиции.

4. Заказы, отмена и история

Чтение заказов

- Получить список своих заказов с фильтром `status=paid` или `cancelled`, пагинацией и итоговыми суммами.
- Получить свой заказ по ID со всеми позициями, снимками цен и названий, `subtotal`, `discount`, `total`, промокодом и временем.
- Получить историю смены статуса. Для нового заказа первая запись - `paid`, для отмененного затем идет `cancelled`.
- Обращение к существующему чужому заказу возвращает 403. Несуществующий заказ возвращает 404. Для корзины, кошелька и истории действует та же проверка владельца.

Отмена: единая транзакция

- Проверить существование заказа, владельца, активность пользователя и статус `paid`. Заблокировать заказ, кошелек и товары.
- Вернуть на склад исходные количества из позиций заказа и вернуть в кошелек ровно `order.total`, то есть фактически списанную сумму со скидкой.
- Записать `refund` в движения кошелька, изменить статус на `cancelled`, добавить историю статуса, операцию отмены и задачу уведомления.
- Корзину не восстанавливать. Промокод после отмены считается использованным: отмена не дает повторную скидку.
- Если заказ уже `cancelled`, новый запрос отмены возвращает 409. Повтор того же запроса с тем же ключом идиempотентности возвращает сохраненный успешный результат без нового возврата.

История пользователя

Храните структурированные записи с ID, `user_id`, типом операции, временем, ID связанной сущности и деталями. Фиксируйте создание, переименование и удаление пользователя, пополнение, изменение корзины и промокода, `checkout` и отмену. Изменение и его история записываются в одной транзакции.

Дайте отдельное чтение движений кошелька и общей истории операций. Для истории поддержите фильтры `type`, `from`, `to` в RFC3339: `from` включительно, `to` исключительно. Неуспешные попытки можно записывать в технический лог, но они не создают успешное движение денег или бизнес-операцию.

Проверка

Повторная или параллельная отмена не возвращает деньги и товары дважды. История и кошелек согласованы с фактическим состоянием заказа.

5. Промокоды и отчеты

В полном проекте эти возможности обязательны, хотя в V2 они предлагались как дополнительные задания.

Промокоды

- Администратор создает код: `code`, `discount_percent` от 1 до 99, `active`. Код длиной 3-32 символа: A-Z, 0-9 и дефис. Нормализуйте пробелы по краям и регистр; уникальность проверяется после нормализации.
- Администратор может включить или отключить код. Процент созданного кода не изменяется. Пользователь применяет один код к своей корзине или удаляет его; новый код заменяет предыдущий.
- Скидка $\text{discount} = \text{floor}(\text{subtotal} * \text{percent} / 100)$, $\text{total} = \text{subtotal} - \text{discount}$. Сначала безопасно вычислите значение, исключив переполнение. Снимки `code`, `percent` и `discount` сохраняются в заказе.
- Код считается использованным только при успешном `checkout`. Один пользователь может успешно использовать один код ровно один раз; другие пользователи могут использовать тот же код.
- Неудачная покупка не расходует код. Конкурирующие покупки не обходят ограничение уникальности (`user_id`, `promo_code_id`). После отмены использование сохраняется.

Отчеты

Отчет	Правила результата
Топ-3 товаров	Сумма <code>quantity</code> по заказам <code>paid</code> , сортировка по количеству по убыванию, при равенстве по <code>product_id</code> . Вернуть ID, текущее название и проданное количество. Если товаров меньше трех, вернуть доступные.
Покупки пользователя	Сумма <code>total</code> по его заказам <code>paid</code> с учетом скидок. Отмененные заказы исключены. Без покупок вернуть 0.
Низкий остаток	Товары со <code>stock <= threshold</code> , включая нулевой остаток. <code>threshold >= 0</code> , по умолчанию 5. Сортировка <code>stock</code> , затем ID; пагинация.

После отмены

Отмененный заказ исключается из суммы покупок и топа продаж, а возвращенные товары учитываются в остатках. Отчеты рассчитываются из актуальных данных БД; отдельную ручную синхронизируемую мапу со статистикой не заводите.

6. Архитектура и организация кода

Часть проекта	Ответственность
cmd/api	Загрузка конфигурации, сборка зависимостей, запуск HTTP и фоновых обработчиков, завершение работы.
internal/domain	Модели, конструкторы, методы, инварианты и доменные ошибки. Без HTTP и SQL.
internal/service	Сценарии пользователя, кошелек, каталога, корзины, заказа, промокодов и отчетов. Управление транзакциями.
internal/storage/postgres	SQL, преобразование строк БД в модели, реализация репозитория и транзакционного интерфейса.
internal/transport/http	Маршруты, JSON DTO, валидация формата, middleware, перевод ошибок в HTTP.
internal/worker	Опрос сохраненных задач, ограниченная очередь, worker pool, повторные попытки и остановка.
migrations, tests	Миграции схемы, интеграционные сценарии и тестовые данные.

Интерфейсы и композиция

- Опишите небольшие интерфейсы хранилищ рядом с сервисами, которые их используют, и интерфейс отправки уведомления Notifier. Передавайте зависимости через конструкторы.
- Для операции из нескольких репозиториях используйте один transaction scope: все SQL-запросы checkout или отмены должны работать через одну и ту же транзакцию.
- Сервисы связывайте через поля и зависимости. Embedding можно использовать для общего технического поведения, если оно действительно нужно; искусственную иерархию сущностей создавать не требуется.
- In-memory реализация допустима как test double. Рабочий сервис хранит данные в PostgreSQL; параллельная production-версия на мапах не требуется.

Ошибки и управление ресурсами

Используйте error, оборачивание причин и errors.Is / errors.As. Разделите неверный ввод, отсутствие, запрет доступа, конфликт состояния и технический сбой. В HTTP не отдавайте SQL, стек и секреты. Ожидаемые ошибки не выражайте через panic.

Используйте defer для освобождения ресурсов и rollback незавершенной транзакции. Recovery на границе HTTP перехватывает непредвиденную панику и пишет технический лог. В worker паника задачи также не должна молча уничтожать обработчик.

7. PostgreSQL: основные таблицы

Реализуйте миграции up/down и отдельный seed для демонстрационных данных. Ниже минимальная логическая схема. Типы и технические поля можно дополнить, сохранив все связи и ограничения.

Таблица	Основные поля и ограничения
users	id PK, name, created_at, updated_at, deleted_at.
wallets	id PK, user_id FK UNIQUE, balance BIGINT CHECK >= 0, updated_at. Один кошелек пользователя.
wallet_transactions	id PK, wallet_id FK, order_id FK nullable, type (top_up / payment / refund), amount BIGINT со знаком, balance_after >= 0, created_at. payment < 0, остальные > 0.
products	id PK, name, price BIGINT CHECK > 0, stock BIGINT CHECK >= 0, created_at, updated_at.
carts	id PK, user_id FK UNIQUE, promo_code_id FK nullable, updated_at. Одна корзина пользователя.
cart_items	cart_id FK, product_id FK, quantity BIGINT CHECK > 0, UNIQUE(cart_id, product_id).
orders	id PK, user_id FK, subtotal > 0, discount >= 0, total > 0, status, promo_code_id FK nullable, promo_code_snapshot, promo_percent_snapshot, created_at, cancelled_at.
order_items	id PK, order_id FK, product_id FK, product_name_snapshot, unit_price BIGINT > 0, quantity BIGINT > 0, UNIQUE(order_id, product_id).
order_status_history	id PK, order_id FK, from_status nullable, to_status, created_at. Первая запись: NULL -> paid.
operation_logs	id PK, user_id FK, operation_type, entity_id nullable, details JSONB, created_at. Детали содержат необходимые снимки операции.

Связи и согласованность

User связан с кошельком и корзиной один к одному, с заказами и историей один ко многим. Cart и Product связаны через cart_items, Order и Product через order_items. Установите FK и CHECK для статусов и денежных сумм: total = subtotal - discount, discount < subtotal.

Финансовую историю и позиции заказа не удаляйте каскадно. Для удаления позиций при очистке корзины каскад допустим. Для payment и refund обеспечьте уникальность (order_id, type), когда order_id задан.

8. PostgreSQL: служебные таблицы и транзакции

Таблица	Основные поля и ограничения
promo_codes	id PK, code UNIQUE, discount_percent CHECK 1..99, active, created_at.
promo_redemptions	id PK, promo_code_id FK, user_id FK, order_id FK UNIQUE, created_at; UNIQUE(user_id, promo_code_id).
notification_jobs	id PK, order_id FK, event_type, payload JSONB, status, attempts, available_at, lease_until, claim_token, last_error, created_at, sent_at; UNIQUE(order_id, event_type).
notification_deliveries	job_id PK/FK, payload JSONB, delivered_at. Учебный получатель уведомления, вставка идемпотентна по job_id.
idempotency_keys	user_id FK, operation, key, request_hash, response_status, response_body JSONB, created_at; UNIQUE(user_id, operation, key).

Блокировки и конкурентный доступ

- Задайте единый порядок блокировок для всех изменяющих сценариев. Например: строка user как блокировка операций пользователя, затем wallet/cart/order, затем products по возрастанию ID, затем promo_code.
- Каждое изменение корзины и checkout сначала блокирует пользователя. Это не дает checkout купить одну корзину дважды и защищает от одновременного редактирования.
- Для остатков и кошелька используйте блокировки строк либо условные UPDATE с проверкой измененных строк. Все проверки состояния, зависящие от записи, повторяются внутри транзакции.
- Синхронизация через sync.Mutex защищает только память процесса. Корректность покупки между несколькими экземплярами сервиса обеспечивает БД.
- Ограничьте длительность транзакций через context. При конфликте, дедлоке или отмене контекста откатывайте всю операцию; не повторяйте отдельный SQL-фрагмент уже нарушенной транзакции.

Индексы

Добавьте индексы для FK и основных чтений: orders(user_id, id), истории(user_id/order_id, created_at, id), wallet_transactions(wallet_id, id), order_items(product_id), products(stock, id), notification_jobs(status, available_at). Проверяйте планы запросов для отчетов на seed с достаточным количеством строк.

В README зафиксируйте уровень изоляции, порядок блокировок и способ обработки конфликтов. Начать можно с READ COMMITTED и явных блокировок строк, но итоговую корректность подтвердите конкурентными тестами.

9. HTTP API: общие правила и основные маршруты

Префикс API: /api/v1. В таблицах U означает /users/{user_id}. Для операций пользователя X-User-ID должен совпадать с user_id. Для чтения и отмены заказа проверяется его владелец. Чтение списка пользователей, управление каталогом и промокодами, складские отчеты требуют X-Admin-Token.

Метод и путь	Тело / результат
POST /users	{name}; 201: профиль, ID, balance=0. Создание доступно в учебном режиме без заголовка пользователя.
GET /users	q, limit, offset; 200: список пользователей (admin).
GET / PATCH / DELETE U	GET: профиль; PATCH {name}: обновленный профиль; DELETE: 204 при выполнении условий удаления.
GET U/wallet	200: user_id, balance.
POST U/wallet/top-ups	{amount}, Idempotency-Key; 200: баланс и ID движения.
GET U/wallet/transactions	limit, offset; 200: движения кошелька.
GET /products	q, limit, offset; 200: товары. GET /products/{id}: один товар.
POST /products	{name, price, stock}; admin; 201: товар.
PATCH /products/{id}	{name?, price?}, хотя бы одно поле; admin; 200: товар.
PUT /products/{id}/stock	{stock}; admin; 200: товар с новым абсолютным остатком.
POST /products/{id}/stock/add	{quantity}; admin; 200: товар с увеличенным остатком.
GET U/cart	200: items, subtotal, promo_code, discount, total.
POST U/cart/items	{product_id, quantity}; 200: обновленная корзина.
PUT U/cart/items/{product_id}	{quantity}; 200: обновленная корзина.
DELETE U/cart/items/{product_id}	204; отсутствующая позиция - 404.
DELETE U/cart	204: корзина и выбранный промокод очищены.

Созданный ресурс возвращайте вместе с ID. GET отсутствующего ресурса - 404. Пустой список - 200 и items: []. В коллекциях возвращайте items, limit, offset; количество всех строк можно не считать.

10. HTTP API: заказы, промокоды и ошибки

Метод и путь	Тело / результат
POST U/checkout	Без тела; Idempotency-Key; 201: полный созданный заказ.
GET U/orders	status?, limit, offset; 200: свои заказы.
GET /orders/{id}	200: полный заказ с позициями и суммами.
POST /orders/{id}/cancel	Без тела; Idempotency-Key; 200: отмененный заказ.
GET /orders/{id}/history	200: история статусов, limit, offset.
GET U/history	type?, from?, to?, limit, offset; 200: операции пользователя.
POST /promo-codes	{code, discount_percent, active}; admin; 201: код.
GET /promo-codes	admin; список кодов, limit, offset.
PATCH /promo-codes/{id}	{active}; admin; 200: код.
PUT / DELETE U/cart/promo	PUT {code}: 200 корзина; DELETE: 204, в том числе если кода уже нет.
GET /reports/top-products	admin; 200: до трех лидеров по продажам.
GET U/purchases/total	200: total по заказам paid.
GET /reports/low-stock	admin; threshold?, limit, offset; 200: товары с низким остатком.

JSON и статусы ошибок

Единый формат: {"error":{"code":"insufficient_funds","message":"Недостаточно средств","request_id":"..."}}. Поле code стабильно для клиента. 400 - неверный JSON, поля, ID или query; 401 - отсутствующий или неверный обязательный заголовок доступа; 403 - чужие данные; 404 - ресурс отсутствует; 409 - конфликт состояния, нехватка средств/остатка, повторное использование промокода; 500 - непредвиденная ошибка; 503 - временная недоступность зависимости.

Ограничьте тело запроса 1 MiB (413 при превышении), требуйте application/json для запросов с JSON (415 иначе). Отклоняйте неизвестные поля, несколько JSON-объектов и лишние данные после объекта. Для PATCH отличайте отсутствующее поле от нулевого значения. Ошибку клиенту возвращайте без внутренних подробностей.

11. Повторные запросы и middleware

Идемпотентность денежных операций

Для пополнения, checkout и отмены обязателен Idempotency-Key длиной 1-128 ASCII-символов без пробелов. Область уникальности: действующий пользователь + тип операции + ключ. request_hash учитывает нормализованное тело и путь, включая ID заказа.

- Первый успешный запрос сохраняет бизнес-результат и ответ в idempotency_keys в одной транзакции.
- Повтор с тем же ключом и теми же параметрами возвращает исходный HTTP-статус и тело, даже если после этого состояние заказа изменилось.
- Повтор того же ключа с другими параметрами возвращает 409 idempotency_conflict. Другой ID заказа при отмене считается другими параметрами.
- Параллельные запросы с одинаковым ключом дают один эффект. Второй после ожидания читает сохраненный результат. Отмена ожидания по context не должна повреждать результат первого.
- Неуспешная транзакция не сохраняет успешный ответ и не расходует ключ. После исправления причины клиент может повторить запрос. Сохраненные ключи в рамках учебного проекта не удаляются.

Middleware и контекст

- Генерация request_id и возврат его в заголовке X-Request-ID и ошибках.
- Структурированный лог: request_id, метод, путь, статус, длительность. Не записывайте токены и полные тела запросов.
- Проверка X-User-ID и X-Admin-Token только там, где они нужны. Токен поступает из конфигурации и не хранится в Git.
- Обработка непредвиденной паники, единые JSON-ошибки, ограничение тела и таймаут запроса.
- Передавайте context от HTTP до сервиса и SQL. Отмена клиентского запроса прекращает незавершенную работу; уже выполненный commit не откатывается. Результат такой операции клиент получает повтором ключа.

Проверка живости

GET /health/live возвращает 200, пока процесс работает. GET /health/ready возвращает 200 при готовности принимать запросы и доступной БД, иначе 503. Перед остановкой readiness переключается в 503.

12. Горутины, очередь и уведомления

После покупки и отмены создавайте уведомления `order_paid` и `order_cancelled` с ID заказа, пользователя, суммой и временем. Надежным источником задач служит `notification_jobs`, записанная в той же транзакции, что и заказ.

Обработка задач

- Диспетчер периодически выбирает готовые `pending`-задачи и задачи с истекшим `lease`. В короткой транзакции захватывает их, например через `FOR UPDATE SKIP LOCKED`, и переводит в `processing` с `lease_until` и новым `claim_token`.
- ID задач поступают в буферизированный канал ограниченного размера. Число `worker` задается конфигурацией, по умолчанию 4; емкость очереди 100. Не запускайте отдельную неограниченную горутину на каждую задачу.
- Worker вызывает `Notifier` с `context` и таймаутом. Учебная реализация вставляет запись в `notification_deliveries` по уникальному `job_id` и пишет структурированный лог. Повторная доставка не создает вторую запись.
- После успеха отметьте задачу `sent`. При временной ошибке увеличьте `attempts` и запланируйте повтор с задержками 1, 2, 4, 8 секунды; после пятой неудачной попытки - `failed`. Состояние доступно через административный `GET /notification-jobs?status=...` с пагинацией.
- `Lease` должен превышать таймаут одной попытки. Завершение или перенос задачи принимается только с актуальным `claim_token`, чтобы старый worker не перезаписал состояние после повторного захвата.

Перезапуск и переполнение

Если очередь заполнена, диспетчер ждет свободное место через `select` с `context` и не захватывает бесконечно новые задачи. Незавершенные задачи остаются в БД и снова доступны после истечения `lease`. Ошибка уведомления не отменяет уже оплаченную покупку.

Доставка имеет семантику как минимум один раз: процесс может завершиться после отправки, но до отметки `sent`. Поэтому получатель идемпотентен по `job_id`. Не обещайте однократный вызов `Notifier`.

Синхронизация и остановка

Примените `select` для отправки/получения и отмены, `WaitGroup` для ожидания worker, `Mutex` или `atomic` для общих счетчиков задач. При остановке сначала прекратите HTTP-прием и дождитесь активных запросов, затем остановите диспетчер, закройте канал со стороны единственного отправителя и дайте worker завершить очередь. По общему дедлайну отмените оставшуюся работу и закройте БД последней.

13. Тесты: бизнес-правила и API

Проверяйте поведение на трех уровнях: модели и сервисы, HTTP через httptest, PostgreSQL на отдельной тестовой БД. Табличные тесты используйте для наборов валидных и невалидных входов. Нужны проверки результата и отсутствия побочных изменений при отказе.

Группа	Обязательные сценарии
Модели	Пустые имена, границы цен/остатков/количества, переполнение денег; успешные методы и отказ без изменения; pointer receiver; копия слайса заказа и независимость возвращенных коллекций.
Пользователи / кошелек	Создание с нулевым балансом; поиск без регистра; переименование; удаление с проверками; пополнение и запись движения; несуществующий пользователь; отрицательная и нулевая сумма.
Каталог / корзина	Поиск; изменение цены и остатка; повторное добавление; SetQuantity=0; отсутствующая позиция; отрицательное количество; очистка; добавление сверх остатка разрешено до checkout.
Покупка	Успех; пустая корзина; недостаток денег или товара; неизменность всех таблиц при отказе; очистка корзины; сохранение снимков после изменения товара.
Отмена	Полный возврат именно уплаченной суммы; восстановление склада; чужой заказ; повторная отмена; отсутствие второго refund; история статусов.
Промокоды / отчеты	Нормализация кода, округление скидки, отключенный код, повторное применение; ошибка покупки не расходует код; отмена не восстанавливает код; топ и суммы исключают cancelled; равные продажи сортируются стабильно.
HTTP	Успех, 400/401/403/404/409/413/415; неизвестные поля; пустые списки; фильтры и пагинация; стабильный формат ошибки и request_id; recovery; отмена context.
Уведомления	Успех, временная ошибка, пять отказов, паника Notifier, повторная доставка, истекший lease, заполненная очередь, остановка без утечки горутин.

Как проверять

Финансовые транзакции и блокировки проверяйте на настоящем PostgreSQL. Моки подходят для изолированной логики и отказов зависимостей, но не подтверждают корректность SQL и конкурентных обновлений.

14. Интеграционные и конкурентные проверки

Сценарий	Ожидаемый результат
Последняя единица	Два пользователя одновременно покупают товар со stock=1. Успешен один заказ; второй получает конфликт; stock=0, списание только у победителя.
Одна корзина	Два checkout одной корзины с разными ключами. Создается один заказ; второй видит пустую корзину. С одинаковым ключом оба получают один результат.
Двойная отмена	Две отмены одновременно. Один возврат и одно восстановление остатков; второй запрос - конфликт либо сохраненный ответ при том же ключе.
Пополнение	Несколько разных пополнений одновременно сохраняют всю сумму. Повтор одного ключа не добавляет сумму повторно.
Обновление склада	Установка остатка или AddStock одновременно с покупкой сериализуются. Итог соответствует одному последовательному порядку операций.
Один промокод	Параллельные попытки не дают одному пользователю две скидки одного кода. Уникальность защищена в БД.
Сбой внутри checkout	Искусственный отказ после части SQL-изменений откатывает кошелек, остатки, заказ, историю, код и задачу уведомления.
Сбой после commit	Повтор запроса с прежним ключом возвращает созданный заказ без нового списания.
Перезапуск worker	Задача processing с истекшим lease подхватывается повторно. В notification_deliveries одна запись.
Два экземпляра	Одновременная обработка API и очереди двумя экземплярами не нарушает ограничения; защита не зависит от локального mutex.

Автоматические команды

Настройте `go test ./...`, `go test -race ./...` и отдельную документированную команду интеграционных тестов. Конкурентные тесты запускают операции через общий барьер; один `sleep` без проверки инвариантов не подтверждает конкурентную корректность.

После сценариев проверяйте: баланс = сумма движений кошелька, нет отрицательных остатков, каждому `paid`/отмененному заказу соответствуют нужные движения и статусы, нет повторного использования кода. Отдельно проверьте корректный `shutdown` с активными запросами и заполненной очередью.

15. Запуск, демонстрация и готовность

Что положить в репозиторий

- Go-модуль, исходники, go.mod/go.sum, .gitignore, Dockerfile, compose.yaml для приложения и PostgreSQL, миграции up/down и отдельный seed.
- README: назначение сервиса, структура, команды запуска/остановки, миграций, seed и тестов; конфигурация, схема таблиц, транзакции, блокировки и повторы задач.
- .env.example с демонстрационными значениями: DATABASE_URL, HTTP_ADDR, ADMIN_TOKEN, таймауты HTTP/SQL/shutdown, число worker, размер очереди, интервал опроса и lease.
- Описание API в OpenAPI и curl-примеры полного сценария. Примеры используют только вымышленные данные и учебные токены.
- CI для форматирования, go vet, unit/race-тестов и интеграционных тестов с PostgreSQL. Желательно разделять функциональность на отдельные коммиты.

Демонстрационный сценарий

Создайте двух пользователей. Пополните первому баланс на 100000. Создайте товар A с ценой 20000 и остатком 5, товар B с ценой 15000 и остатком 3. Добавьте 2 A и 1 B в корзину. Примените промокод на 10%.

Оформите заказ: subtotal=55000, discount=5500, total=49500, баланс=50500, остатки A=3 и B=2. Корзина пустая, в истории есть покупка, уведомление доставлено. Повтор checkout с тем же ключом возвращает этот же заказ.

Измените цену A и убедитесь, что заказ сохранил старую цену. Попытка отмены вторым пользователем дает 403. Отмените заказ первым: баланс снова 100000, остатки A=5 и B=3. Повторная отмена с новым ключом дает 409, повтор с исходным ключом возвращает сохраненный результат. Повторная покупка с тем же промокодом запрещена.

Проект готов, когда

- С чистого checkout репозитория проект запускается по README, применяет миграции и проходит демонстрационный сценарий.
- Все функции доступны через API, сохраняются в PostgreSQL и покрыты перечисленными проверками.
- Работают конкурентные покупки, отмены, восстановление фоновых задач и корректная остановка. go test -race не находит гонок в выполняемых тестах.
- Нет заглушек в обязательных сценариях. Архитектурные решения и ограничения описаны так, чтобы другой разработчик мог продолжить работу.